

P1496R0

Formatting of negative zero

Draft Proposal, 2019-02-20

This version:

<http://fmtlib.net/D1496R0.html>

Authors:

[Alan Talbot](#)

[Jorg Brown](#)

[Victor Zverovich](#)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Draft Revision:

0

Source:

github.com/fmtlib/blob/master/source/D1496R0.bs

Table of Contents

1 Introduction

2 Status quo

3 Suppressing the output of `'-'`

4 Options

References

Informative References

§ 1. Introduction

During the Library Evolution Working Group review of [\[D0645R6\]](#), it has been suggested that formatting of negative floating point zero should be clarified. This is done in the current paper.

§ 2. Status quo

Floating point formatting in [\[D0645R6\]](#) is expressed in terms of `to_chars` ([\[charconv.to.chars\]](#)) and the latter is expressed in terms of `printf`:

`value` is converted to a string in the style of `printf` in the `"C"` locale.

[\[N1256\]](#) doesn't appear to explicitly specify whether `'-'` should be output for negative zero. The current implementation behavior is to print `'-'` for `-0.0` with MSVC `to_chars`, libc `printf`, and the `{fmt}` library that implements [\[D0645R6\]](#). For example, both

```
to_chars(begin, end, -0.0);
```

and

```
printf("%g", -0.0);
```

produce `"-0"`.

§ 3. Suppressing the output of `'-'`

In a use case Alan is familiar with the software generates dozens of tabular reports, each of which has dozens or hundreds of columns, most of which display floating point numbers. The consumers of these reports do not care about or understand negative zero, and will complain if some zeros have a minus sign. With existing output options, negative zeros appear a significant percentage of the time for no reason which a user of the software would understand. It is therefore necessary to do a floating point test for negative zero and change the floating point value in every conversion from floating point to text. This of course has been factored out into a home-grown formatting facility, which is a source of subtle bugs.

With a standard text formatting facility, `std::format` can be used to convert from floating point to text with a formatting string determined in one place by various user settings. If `std::format`

supports the notion of suppressing negative zero, then it is a simple matter of adding that flag to the number format string (probably in one place in the code). It can also trivially be a user selection. None of the hundreds of places where the conversion is actually done needs to know about negative zero, so no home-grown facility is needed.

To address this use case we could add a new format specifier to suppress the output of `'-'` for negative zero, e.g. `'z'` (subject to bikeshedding):

```
string s1 = format("{0:z} {0:+z} {0:-z} {0: z}", 0.0); // s1 == "0 +0 0  
string s2 = format("{0:z} {0:+z} {0:-z} {0: z}", -0.0); // s2 == "0 +0 0
```

in addition to

```
string s3 = format("{0} {0:+} {0:-} {0: }", 0.0); // s3 == "0 +0 0 0"  
string s4 = format("{0} {0:+} {0:-} {0: }", -0.0); // s4 == "-0 -0 -0 -0"
```

It is possible to achieve similar functionality with [\[D0645R6\]](#) by providing a custom formatter:

```

struct dbl {
    double value;
    dbl(double val) : value(val) {}
};

template <>
struct formatter<dbl> : formatter<double> {
    bool allow_negative_zero = true;

    auto parse(format_parse_context& ctx) {
        auto it = ctx.begin(), end = ctx.end();
        if (it != end && *it == 'z') {
            allow_negative_zero = false;
            ++it;
        }
        ctx.advance_to(it);
        return formatter<double>::parse(ctx);
    }

    auto format(dbl d, format_context& ctx) {
        if (!allow_negative_zero)
            d.value -= -0.0;
        return formatter<double>::format(d.value, ctx);
    }
};

string s1 = format("{0:z} {0:z+} {0:z-} {0:z }", dbl(0.0)); // s1 == "0.0
string s2 = format("{0:z} {0:z+} {0:z-} {0:z }", dbl(-0.0)); // s2 == "0.0

```

It is also possible to implement a custom format function that has such behavior by default:

```

template <typename T> const T& maparg(const T& value) { return value; }
dbl maparg(double value) { return value; }

template <typename... Args>
string myformat(string_view fmt, const Args&... args) {
    return format(fmt, maparg(args)...);
}

string s3 = myformat("{0:z} {0:z+} {0:z-} {0:z }", 0.0); // s3 == "0.0 +0.0"
string s4 = myformat("{0:z} {0:z+} {0:z-} {0:z }", -0.0); // s4 == "0.0 +0.0"

```

§ 4. Options

1. Don't do anything: `std::format` will produce `'-'` for `-0.0` whenever `std::to_chars` does.
2. Add a format specifier to suppress the output of `'-'` for `-0.0`.

§ References

§ Informative References

[D0645R6]

Victor Zverovich. [Text Formatting](#). URL:

<http://wiki.edg.com/pub/Wg21kona2019/P0645/D0645R6.html>

[N1256]

[WG14/N1256 Committee Draft](#). URL: <http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1256.pdf>